

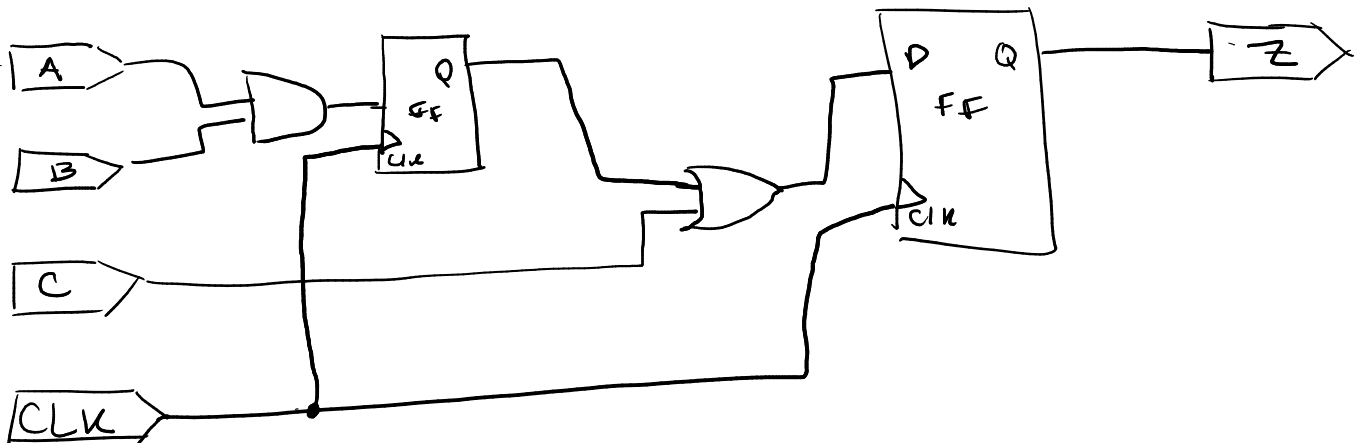
Práctico 1 (Ejs. para parcial) - AdC

Procesador de un ciclo

Ejercicio 9:

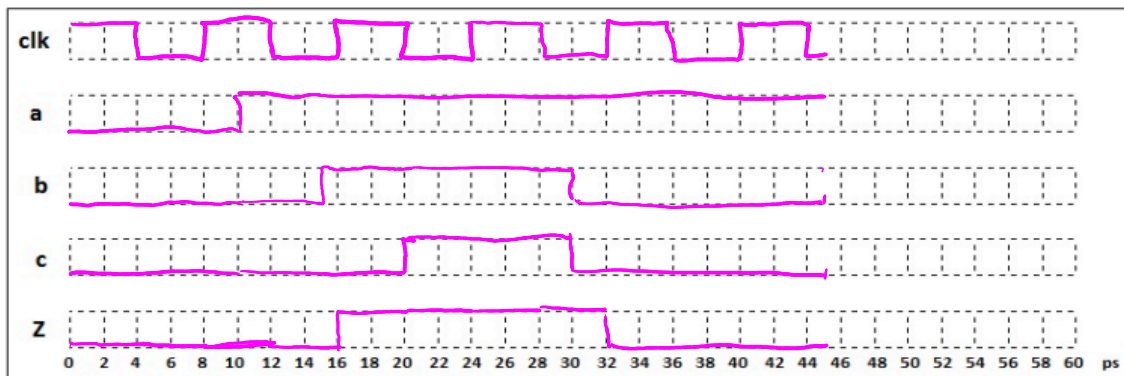
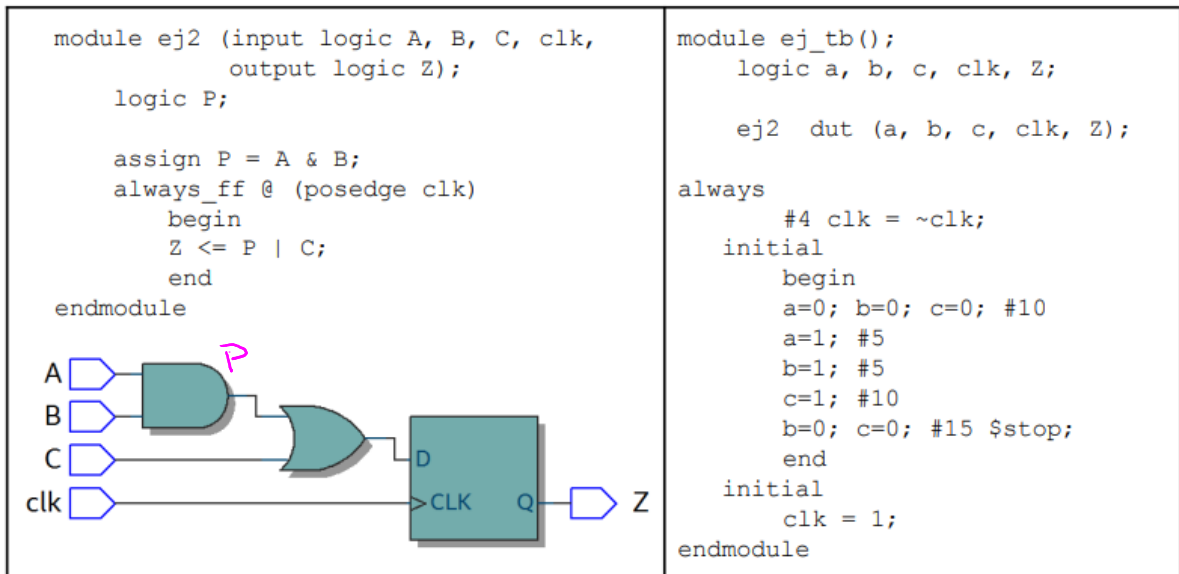
Dibujar el circuito resultante de la síntesis del siguiente código en SystemVerilog:

```
module ej1 (input logic A, B, C, clk,  
            output logic Z);  
    logic P;  
    always_ff @ (posedge clk)  
    begin  
        P <= A & B;  
        Z <= P | C;  
    end  
endmodule
```



Ejercicio 10:

La síntesis del siguiente código en SystemVerilog (**ej2**) da como resultado el circuito de la figura. Dibujar en el gráfico las señales de entrada y salida que resultan de correr el test bench (**ej_tb**) descrito.



Ejercicio 11:

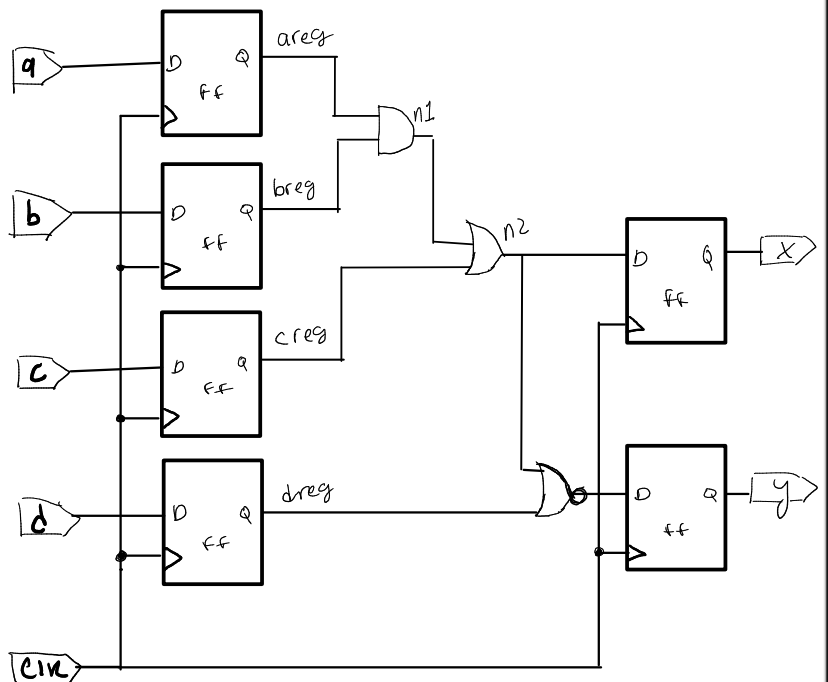
Dibujar en el recuadro el circuito resultante de la síntesis del siguiente código en SystemVerilog:

```

module Recuperatorio
    (input logic clk,a, b, c, d,
     output logic x, y);
    logic n1, n2;
    logic areg, breg, creg, dreg;

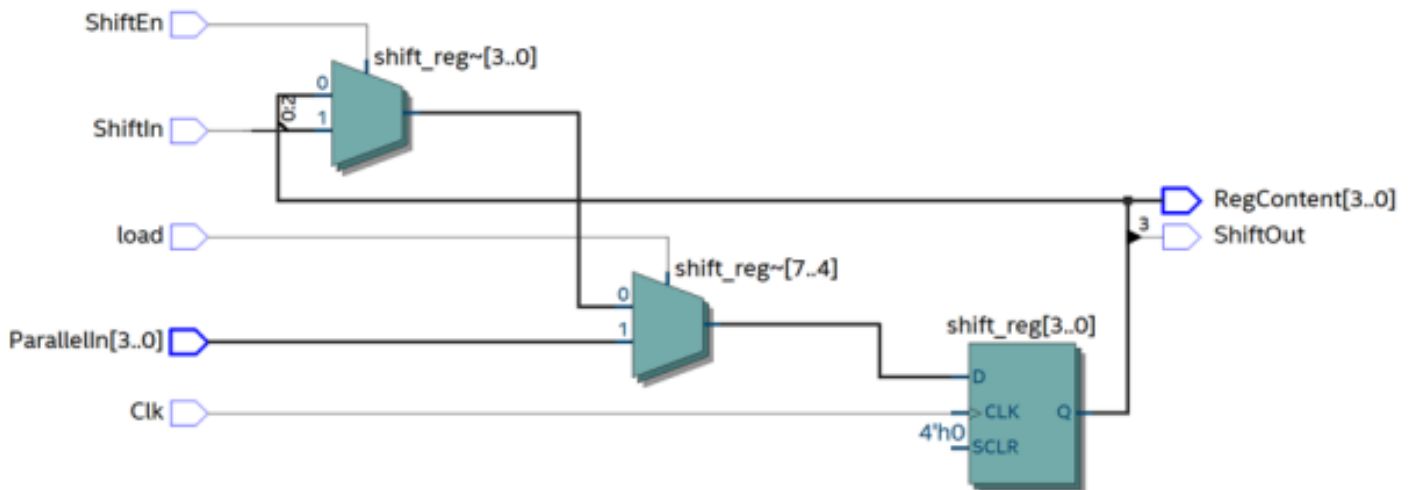
    always_ff @(posedge clk) begin
        areg <= a;
        breg <= b;
        creg <= c;
        dreg <= d;
        x <= n2;
        y <= ~(dreg | n2);
    end
    assign n1 = areg & breg;
    assign n2 = n1 | creg;
endmodule

```



Ejercicio 12:

A continuación se describe en SystemVerilog el módulo **regPS**. Completar (en caso que corresponda) los cuadros vacíos para su correcta interpretación, considerando que el circuito resultante es el que se muestra en la figura.



```
module regPS (input logic Clk, ShiftIn, load, ShiftEn,  
              input logic [3:0] ParallelIn,  
              output logic ShiftOut,  
              output logic [3:0] RegContent);
```

```
    logic [3:0] shift_reg;
```

```
    always-ff @(posedge Clk)
```

```
    if(load)
```

```
        shift_reg <= ParallelIn;
```

```
    else if (ShiftEn)
```

```
        shift-reg[3:0] <= {shift_reg[2:0], ShiftIn};
```

```
    always-comb
```

```
    begin
```

```
        Shift Out = shift_reg[3];
```

```
        RegContent = shift_reg;
```

```
    end
```

```
    end module
```

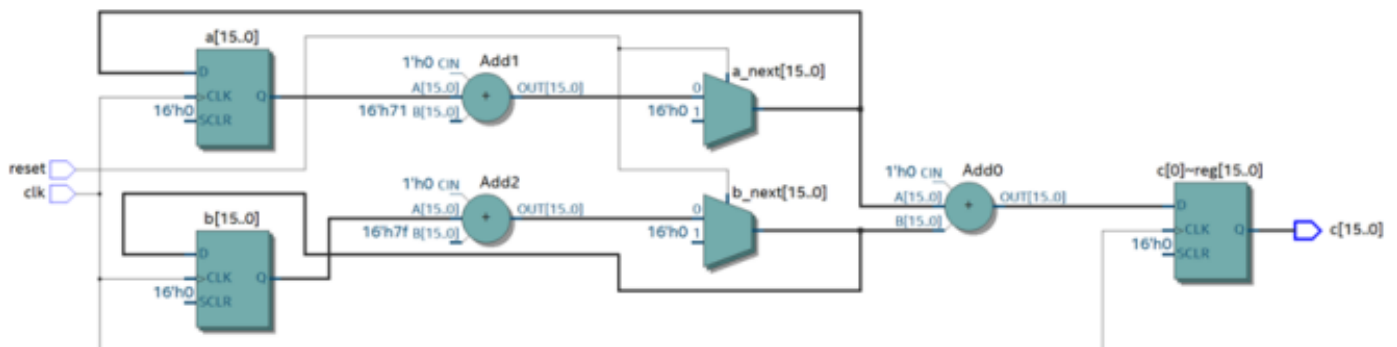
Ejercicio 13:

Dado el siguiente módulo descrito en System Verilog:

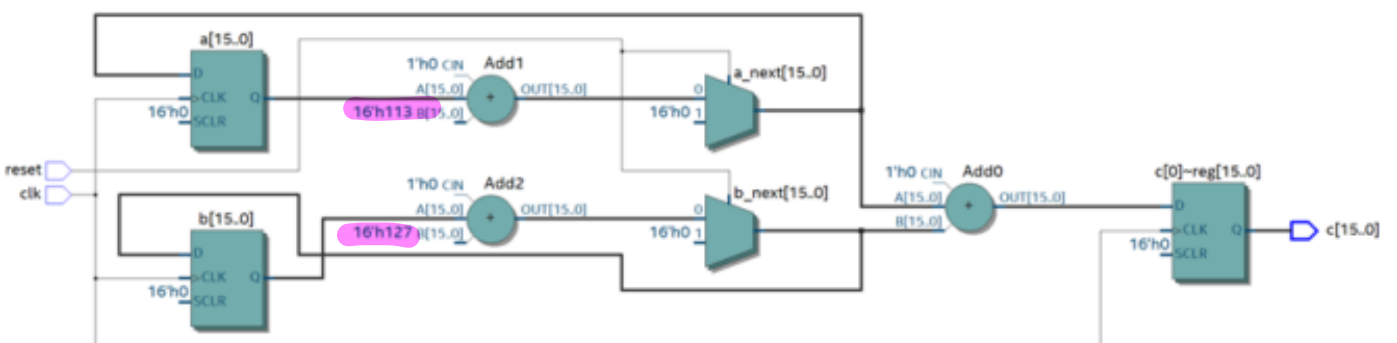
```
module mksum (output logic [15:0] c,  
              input logic reset, clk ) ;  
  
  logic [15:0] a, a_next, b, b_next ;  
  always_ff@(posedge clk)  
  begin  
    a <= a_next ;  
    b <= b_next ;  
    c <= a_next + b_next ;  
  end  
  always_comb  
  begin  
    if ( reset )  
    begin  
      a_next = '0 ;  
      b_next = '0 ;  
    end else begin  
      a_next = a + 16'd113 ;  
      b_next = b + 16'd127 ;  
    end  
  end  
endmodule
```

Seleccione cuál de los siguientes circuitos corresponde a la implementación del módulo **mksum**:

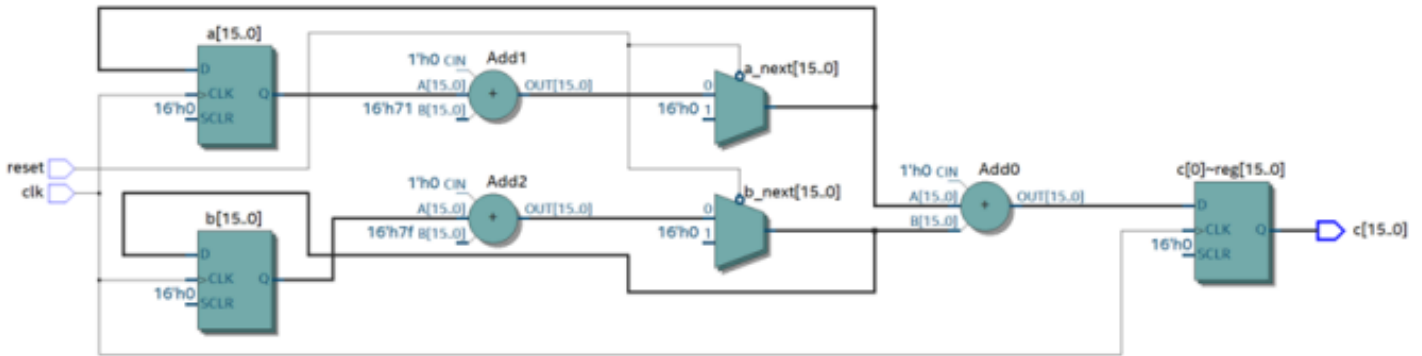
a.



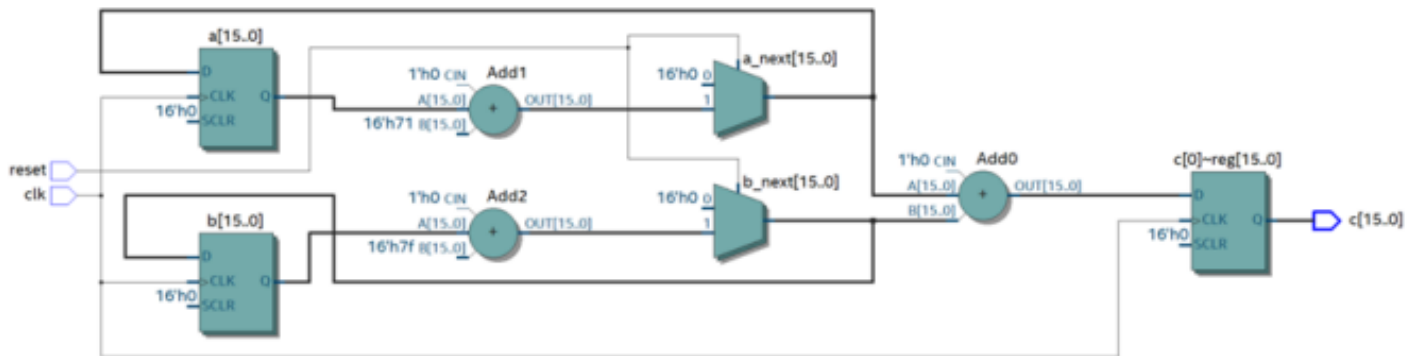
b.



c.



d.



Ejercicio 14:

Dado el siguiente segmento de código en SystemVerilog, con entradas "x*", salidas "z*" y señales internas "y*", seleccione todas las respuestas que considere correctas.

```

1> always_ff @ (posedge clk)
2>   begin
3>       z0 = y0; y0 = x0;
4>       y1 = x1; z1 = y1;
5>       z2 <= y2; y2 <= x2;
6>       y3 <= x3; z3 <= y3;
7>   end

```

Handwritten annotations in pink:

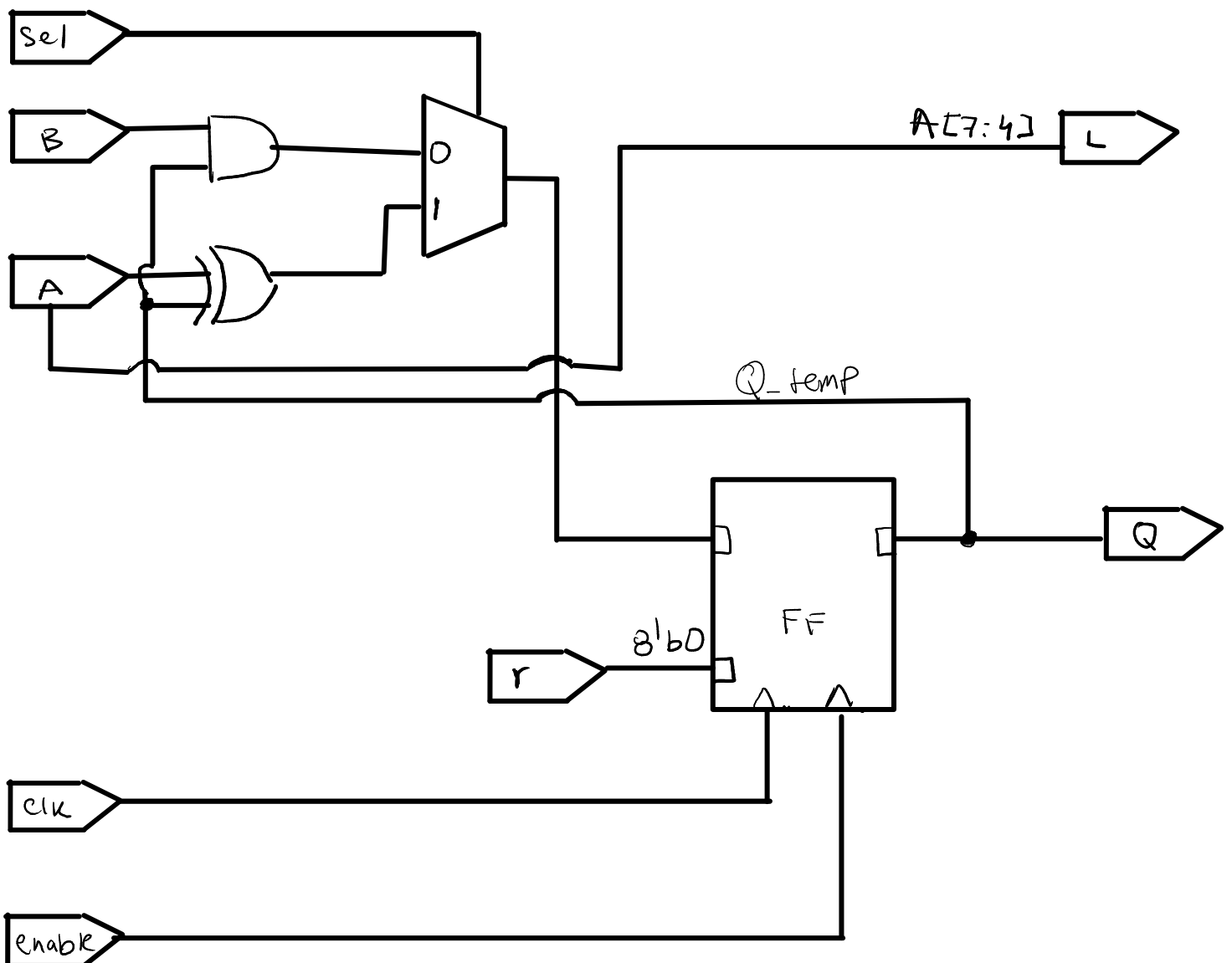
- For line 3: $y_0 \rightarrow z_0$ and $x_0 \rightarrow y_0$ are labeled "paralelo".
- For line 4: $x_1 \rightarrow y_1$ and $y_1 \rightarrow z_1$ are labeled "paralelo".
- For line 5: $x_2 \rightarrow y_2 \rightarrow z_2$ is labeled "desplazamiento".
- For line 6: $x_3 \rightarrow y_3 \rightarrow z_3$ is labeled "desplazamiento".

- ☒ a) La línea 3 implementa un registro paralelo.
- ☐ b) La línea 5 implementa un registro paralelo.
- ☒ c) La línea 6 implementa un registro de desplazamiento (shift register).
- ☐ d) La línea 4 implementa un registro de desplazamiento (shift register).

Ejercicio 15:

a) Dibujar el circuito resultante de la síntesis del siguiente código en SystemVerilog:

```
module circuito
  (input logic clk, r, enable, sel,
   input logic [7:0] A, B,
   output logic [7:0] Q,
   output logic [3:0] L);
  logic [7:0] Q_temp;
  always_ff @ (posedge clk, negedge r)
  begin
    if (~r)
      Q_temp <= 8'b0;
    else if (enable)
      if (sel)
        Q_temp <= Q_temp ^ A;
      else
        Q_temp <= Q_temp & B;
  end
  assign L = A[7:4];
  assign Q = Q_temp;
endmodule
```



b) A continuación se describe en SystemVerilog el test bench del módulo **circuito**. Completar las líneas con los elementos faltantes para obtener las formas de onda de respuesta que se muestran en la figura.

```

module circuito_tb();
    logic      clk, reset, enable, s;
    logic [3:0] L;
    logic [7:0] a, b, q;
    circuito dut (.clk(clk), .r(reset), .enable(enable));
    always      • sel (s), .A(a), .B(b), .L(L), .Q(q));
        begin
            clk = 0 ; #100; clk = 1 ; #100;
        end
    initial
        begin
            reset = 0; enable = 0; s = 1; #250 ;
            reset = 1;
            a = 8'b01010101;
            b = 8'b11111111 ; #250 ;
            enable = 1; #50 ;
            s = 0; #200 ;
            $stop;
        end
    endmodule

```

